Classes and Objects:

Types of classes in JAVA: Classes in JAVA classified into

i. Top-level class – It is a class that is not nested class
ii. Nested class – It is a class that is declared with in a body of another class or an interface. Nested classes are again classified into:
    a. Nested static class - A static class that is created inside a class, is called a static nested class in Java.
    b. Nested non-static class also called as inner classes classified into:
        i. Local class – is defined with in a method or code block
        ii. Anonymous class – It is the inner class without name
iii. Derived class or subclass – A class derive from another class
iv. Super class – It is a class, from which another classes are derived
v. Generic class - A **class** that can refer to any type is known as a **generic class**.
vi. Final class – It is a class that cannot be inherited.

Class Declaration and Modifiers:

A **class** is a fundamental construct used to define a new data type. It serves as a blueprint for creating objects, which are instances of the class. A class encapsulates data for the object and methods to manipulate that data.

Syntax:

```
modifier class classname
{
        type instance-variable1;
        type instance-variable2;
        // ...
        type instance-variableN;
        type methodname1(parameter-list) {
                // body of method
        }
        // ...
        type methodnameN(parameter-list) {
                // body of method
        }
}
```

Ex:

```
public class Person {
        private String name;
        private int age;

        public Person(String n, int a) {
```

```
            name = n;
            age = a;
        }

        public void displayInfo() {
            System.out.println("Name: " + name);
            System.out.println("Age: " + age);
        }

        public void setName(String name) {
            this.name = name;
        }
    }
```

A Class in Java can contain:

- Data member
- Method
- Constructor
- Nested Class
- Interface

```
access_modifier class <class_name>
{
   data member;
   method;
   constructor;
   nested class;
   interface;
}
```

Naming conventions that are generally used to create classes in JAVA :

- The class name should be simple and descriptive
- Names should start with upper-case letter
- Reserved words should be avoided.

Modifiers: Class modifiers help to control the access to class, constructor, variable, method, or data member. It provides security, accessibility, etc to the user depending upon the access modifier used with the element.

The modifiers are classified into:

i. Access modifiers ii. Non-access modifiers

Access modifiers:

i. **default (No modifier)** - If no access specifier is provided, the member is accessible only within the same package. This is also known as package-private access.

Ex:

```
class MyClass {
   int field;
   void methodName() {      // Method code      }
}
```

ii. **public** - The member is accessible from any other class, regardless of the package.
Ex:

```
public class MyClass {
  public  int field;
   public void methodName() {          // Method code       }
}
```

iii. **private** - The member is accessible only within the same class. It cannot be accessed from outside the class, even by subclasses.
Ex:

```
class MyClass {
   private int field;
   void methodName() {          // Method code       }
}
```

iv. **protected** - The member is accessible within the same package and by subclasses, even if they are in different packages.
Ex:

```
class MyClass {
  protected int field;
   void methodName() {          // Method code       }
}
```

Non-access modifiers:

i. final - The final modifier is used to define constants, prevent method overriding, and prevent inheritance of classes.

Ex:

```
  final class MyClass {
}
```

ii. abstract - The abstract modifier is used to define abstract classes and methods.

Ex:

```
  abstract class MyClass {
}
```

iii. **strictfp -** The strictfp modifier is used to ensure that floating-point calculations adhere to the IEEE 754 standard for floating-point arithmetic, making them consistent across different platforms.

Ex:

```
public strictfp class StrictfpExample {
    public strictfp double calculate(double a, double b) {
    return a / b; // Calculations are performed in strict floating-point mode
  } }
```

Class Members:

- The class members are declared in the body of a class.
- These may comprise fields (variables in a class), methods, nested classes, and interfaces.
- The members of a class comprise the members declared in the class as well as the members inherited from a super class.
- Types of variables in class:

  Non-static variables or Non Class variables: These include local and instance variables and varies in scope and value.

  i. *Instance variables*: These variables are individual to an object and an object keeps a copy of these variables in its memory. These are initialized when an object is created.
  ii. *Local variables*: These are local in scope and not accessible outside their scope.

  Class or static variables: These variables are also qualified as **static variables.**

  a) The values of these variables are common to all the objects of the class.
  b) The class keeps only one copy of these variables and all the objects share the same copy.
  c) As class variables belong to the whole class, these are also called class variables.
  d) These are initialized when class is loaded into JVM memory.

Ex:

```
class Local_Instance_Static {

    // Static variable: Shared among all instances
    private static int staticCount = 0;

    // Instance variable: Unique to each instance
    private int instanceCount = 0;

    // Constructor
    public SimpleVariableDemo() {
        // Increment static variable
        staticCount++;

        // Increment instance variable
        instanceCount++;
    }

    // Instance method to display counts
    public void displayCounts() {
        // Local variable: Exists only within this method
```

```
F:\JAVA2024-25\Unit 2>java Local_Instance_Static
Static Count: 2
Instance Count: 1
Local Count: 5
Static Count: 2
Instance Count: 1
Local Count: 5

F:\JAVA2024-25\Unit 2>
```

```
        int localCount = 5;
        // Display values
        System.out.println("Static Count: " + staticCount);
        System.out.println("Instance Count: " + instanceCount);
        System.out.println("Local Count: " + localCount);
    }

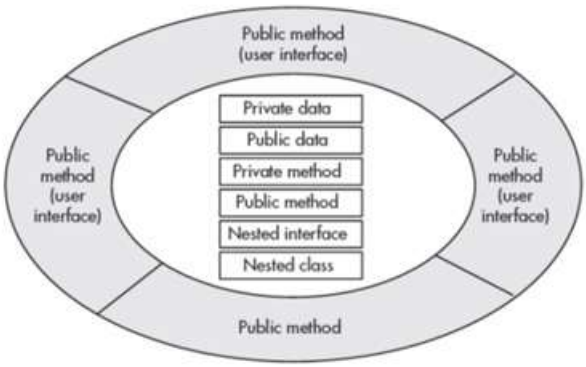
    // Main method
    public static void main(String[] args) {

        SimpleVariableDemo obj1 = new SimpleVariableDemo();
        SimpleVariableDemo obj2 = new SimpleVariableDemo();

        // Call instance method to display counts for each object
        obj1.displayCounts();
        obj2.displayCounts();
    }
}
```

Access Specifiers and Information Hiding:

- A private member can be accessed by the other members of the same class.
- A public member can be accessed by any one.
- *All the private members can be accessed through the public methods of the same class*



Declaration of Class Objects:

The declaration of an object is similar to declaration of primitive type variable:

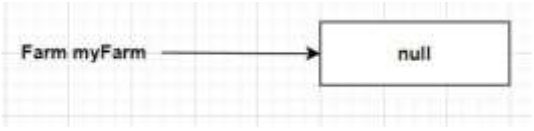
class_name object_identifier;

Ex:

```
public class Farm{
  int length;
  int width;
}
```

An object of Farm class can be created as

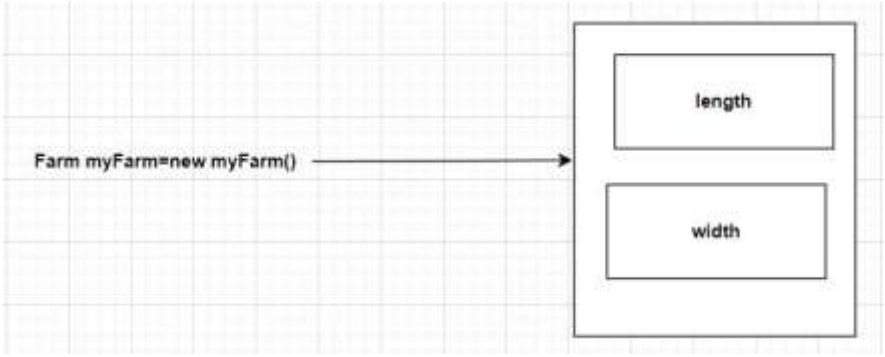
Farm myFarm;



Here, the variable *myFarm* simply refers to an object of Farm class whose value at present is *null.* The object comes into existence only when a physical copy of object of class (i.e., required memory) is assigned with the operator *new.*

The *new* operator creates a new object or instance of class. In practice, this dynamically allocates memory in the heap at run time of the program.

Farm myFarm = new Farm();



From the above figure, each object of the *Farm* class has allocates the memory that van hold the values for instance variables(length, width).

Ex:

```
class Farm
{
        double length;
        double width;
}
class FarmEx
{
        public static void main(String[] arg)
        {
                Farm myFarm=new Farm();
```

```
                myFarm.length=10.0;
                myFarm.width=10.0;
                System.out.println("Area:"+myFarm.length*myFarm.width);
        }
}
```

Assigning one object to another object:

In JAVA, it is possible to copy the value of one object to another. This can be done in different ways:

i. Using the assignment operator (=): If the objects are of the same class type, you can simply assign one object to another using the assignment operator. This will create a reference to the same object in both variables.

```
MyClass obj1 = new MyClass();
MyClass obj2 = obj1; // obj2 now references the same object as obj1
```

ii. Using a copy constructor: If the class has a copy constructor, you can create a new object by passing the original object to the copy constructor.

```
MyClass obj1 = new MyClass(someData);
MyClass obj2 = new MyClass(obj1); // obj2 is a new object with the same data as
    obj1
```

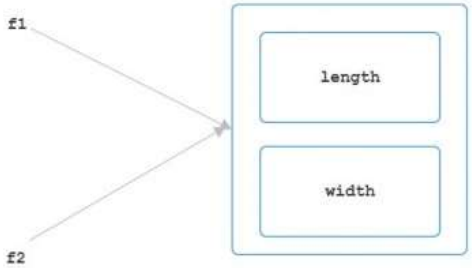
iii. Using a clone() method: If the class implements the Cloneable interface and provides a clone() method, you can create a new object by calling the clone() method on the original object.

```
MyClass obj1 = new MyClass(someData);
MyClass obj2 = (MyClass) obj1.clone(); // obj2 is a new object with the same
    data as obj1
```

Using the assignment operator (=):

When one object of a class provides a reference to another object of the same class, the first object can access the second object's data and methods.

```
class Farm
{
        double length;
        double width;
        double area(){return
length*width;}
}
```



```
class ObjectReference
{
	public static void main(String[] arg)
	{
		Farm f1=new Farm();
		Farm f2;	//=new Farm();
		f1.length=10.0;
		f1.width=10.0;
		System.out.println("Object f1\'sArea:  "+f1.area());

		f2=f1;

		System.out.println("Object f2\'s Area:  "+f2.area());

		f2.length=20;
		System.out.println("Object f1\'sArea:  "+f1.area());
		System.out.println("Object f2\'s Area:  "+f2.area());
	}
}
```

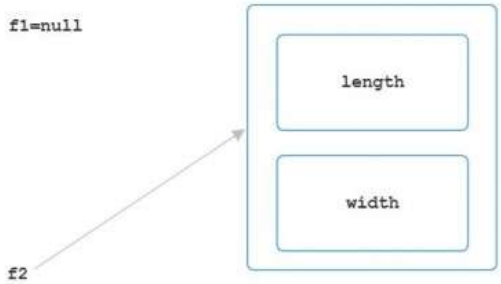
Output:

```
F:\JAVA2024-25\Unit 2>java ObjectReference
Object f1'sArea:  100.0
Object f2's Area:  100.0
Object f1'sArea:  200.0
Object f2's Area:  200.0
```

In the above both objects f1 and f2 refers to the same memory locations which is allocated to f1. If you change the content of f1's object it will be reflected to f2 and if you change the object f2 values it also reflected to f1.

Another Example:

```
class ObjectReferenceAnother
{
	public static void main(String[] arg)
	{
		Farm f1=new Farm();
		Farm f2;
		f1.length=10.0;
		f1.width=10.0;
		System.out.println("Object f1\'sArea:  "+f1.area());
		f2=f1;
```



```
            System.out.println("Object f2\'s Area:  "+f2.area());
            f1=null;
            System.out.println("Object f2\'s Area:  "+f2.area());
            System.out.println("Object f1\'sArea:  "+f1.area());
        }
}
```

Output:

```
F:\JAVA2024-25\Unit 2>javac ObjectReferenceAnother.java

F:\JAVA2024-25\Unit 2>java ObjectReferenceAnother
Object f1'sArea:  100.0
Object f2's Area:  100.0
Object f2's Area:  100.0
Exception in thread "main" java.lang.NullPointerException:
Cannot invoke "Farm.area()" because "<local1>" is null
        at ObjectReferenceAnother.main(ObjectReferenceAnoth
er.java:23)

F:\JAVA2024-25\Unit 2>
```

Constructor and Methods:

Defining Methods:

A method definition comprises two components:

I. *Header* that includes modifier, type, identifier or name of method and a list of parameters. The parameter list is placed in a pair of parentheses.

II. *Body* that is placed in braces ({ }) and consists of declarations and executable statement and other expressions.

Syntax:

```
modifier type method_name(parameter-list) [throws exception_list]
{
        // body of method
}
```

a) Modifiers for a method:

There a several modifiers that may be part of a method declaration:

- Access modifiers: **public, protected, and private**
- Non – access Modifiers:

✓ **static:** Attributes and methods belongs to the class, rather than an object
✓ **final:** Methods cannot be overridden/modified

- ✓ **abstract:** The method does not have an implementation in the class where it is declared. It must be implemented by subclasses. Abstract methods are used in abstract classes.
- ✓ **synchronized:** Ensures that a method can only be executed by one thread at a time. It is used in multi-threaded programming to control access to critical sections of code.
- ✓ **native** - Indicates that the method is implemented in native code using JNI (Java Native Interface). It allows Java code to call functions written in other languages like C or C++.
- ✓ **strictfp** - Modifier forcing strict floating point behaviour
- ✓ **volatile** - The value of an attribute is not cached thread-locally, and is always read from the "main memory" –Applicable to variable/fields
- ✓ **transient –** Applicable to fields/variables - In Java, the transient keyword is used to indicate that a field of a class should not be serialized. Serialization is the process of converting an object's state to a byte stream so that it can be easily saved to a file or transmitted over a network. When an object is serialized, all of its non-transient fields are included in the serialized form. Fields marked as transient are excluded from this process.

b) Method type: The *type* of a method is same as the *type* of data it returns.

For ex:

- If it returns int number, its *type* is int;
- if it returns a double number, its *type* is double;

c) Identifier:

- The name or an identifier of a user defined method is chosen in the same way as choosing name of a variable.
- The identifier or the method name should be a valid identifier in Java.
- It should not be a keyword.
- It should not start with a digit.
- It should be a unique name in its scope and not a duplicate name of a predefined variable or constant or method of the class.

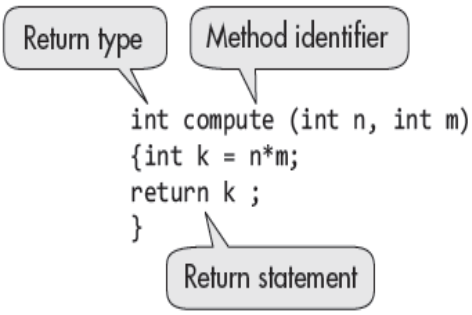
d) Parameter list:

- The parameter list is enclosed in a pair of parentheses.
- Each parameter is declared with its *type* individually and separated by a comma from others as

myMethod (int n, int m, fl oat x, double y)

e) Return statement:

- In the case of methods of *type* void no return statement is required.
- In return type methods, the last statement is the return statement, that is, return *value*;



Example:

```
class Box
{
        double width;
        double height;
        double depth;
        void setDimensions(double w, double h, double d)
        {
                width=w;
                height=h;
                depth=d;
        }
        // display volume of a box
        void volume() {
                System.out.print("Volume is ");
                System.out.println(width * height * depth);
        }
}
class Method_Example {
        public static void main(String args[])
        {
                Box mybox1 = new Box();
                Box mybox2 = new Box();
                // assign values to mybox1's instance variables
                mybox1.setDimensions(10,20,15);
                /* assign different values to mybox2's instance variables */
                mybox2.setDimensions(3,6,9);
```

```
                // display volume of first box
                mybox1.volume();
                // display volume of second box
                mybox2.volume();
        }
}
```

Output:

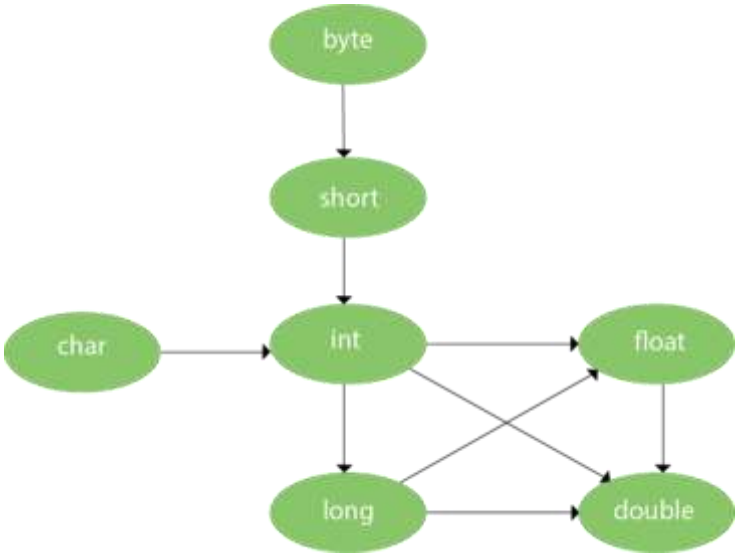
```
Volume is 3000.0
Volume is 162.0
```

Overloaded Methods:

Method overloading in Java is a feature that allows a class to have more than one method with the same name, but with different parameters. This is useful for creating methods that perform similar functions but with different types or numbers of arguments.

How Method Overloading Works

- **Different Parameters**: The methods must have different parameter lists. This means they can differ in:
    - **Number of parameters**: E.g., void method(int a) and void method(int a, int b)
    - **Type of parameters**: E.g., void method(int a) and void method(double a)
    - **Order of parameters**: E.g., void method(int a, double b) and void method(double a, int b)
- **Same Method Name**: All overloaded methods share the same name but must have different parameter lists.
- **Return Type**: The return type alone is not sufficient to distinguish overloaded methods. The parameter list must be different.
- **Compile-Time Polymorphism**: Method overloading is a type of compile-time polymorphism. The correct method is determined at compile time based on the method signature. Also called as static time binding.
- If there is no exact matching of parameters – type matching can be performed.



Ex:

```
class OverloadExample {
   // Method with one int parameter
   public void display(int a) {
      System.out.println("Display method with int parameter: " + a);
   }
   // Method with two int parameters
   public void display(int a, int b) {
      System.out.println("Display method with two int parameters: " + a + ", " + b);
   }
   // Method with one double parameter
   public void display(double a) {
      System.out.println("Display method with double parameter: " + a);
   }
   // Method with one int and one double parameter
   public void display(int a, double b) {
      System.out.println("Display method with int and double parameters: " + a + ", " + b);
   }
}
class MethodOverload{
            public static void main(String[] args) {
      OverloadExample obj = new OverloadExample();
      obj.display(5);           // Calls method with int parameter
      obj.display(5, 10);       // Calls method with two int parameters
      obj.display(5.5);         // Calls method with double parameter
      obj.display(5, 5.5);      // Calls method with int and double parameters
   }
}
```

Output:

```
Display method with int parameter: 5
Display method with two int parameters: 5, 10
Display method with double parameter: 5.5
Display method with int and double parameters: 5, 5.5
```

Example program on method overloading if there is no exact match:

```
class OverloadExample {
    // Method with one double parameter
   public void display(double a) {
      System.out.println("Display method with double parameter: " + a);
   }

}
class MethodOverload{
            public static void main(String[] args) {
      OverloadExample obj = new OverloadExample();
      obj.display(5);           // Calls method with int parameter
       obj.display(5.5);         // Calls method with double parameter
    }
}
```

Output:

```
Display method with double parameter: 5.0
Display method with double parameter: 5.5
```

Constructors:

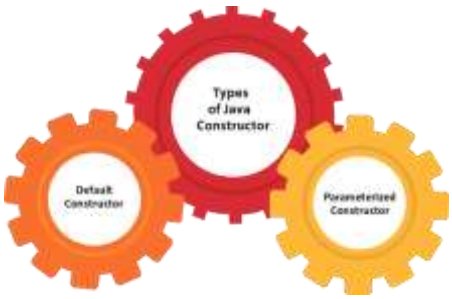
In Java, a constructor is a special method used to initialize objects. Unlike regular methods, constructors have the same name as the class and do not have a return type, not even void. The constructor is called when an object of a class is created. It can be used to set initial values for object attributes.

Key Features of Constructors

1. **Name**: The name of the constructor must be the same as the class name.
2. **No Return Type**: Constructors do not have a return type, not even void. By default the return type is class type.
3. **Initialization**: Constructors are used to initialize the object's state by assigning values to its fields.

4. **Default Constructor**: If no constructor is defined, Java provides a default constructor with no parameters that initializes the object with default values.

Types of Constructors:



- **Default Constructor**: A constructor without any arguments is called default or no argument constructor.

  If no constructor is defined, Java provides a default constructor with no parameters that initializes the instance variables with default values.



- **Parameterized Constructor**: A constructor that takes arguments to initialize the object with specific values.

Ex: Default Constructor or no argument Constructor:

```
class Farm
{
        double length;
        double width;
        Farm()
        {
                length=10;
                width=10;
        }
        double area() {
                return (width * length);
        }
}
class DefaultConstructor {
        public static void main(String args[])
        {
                Farm f = new Farm();
                System.out.println("Area is: "+f.area());

        }
```

```
}
```

Output: Area is: 100.0

Example program on Parameterized Constructor:

```
class Farm
{
        double length;
        double width;
        Farm(double length, double width)
        {
                this.length=length;
                this.width=width;
        }
         double area() {
                return (width * length);
        }
}
class ParameterizedConstructor {
        public static void main(String args[])
        {
                Farm f = new Farm(10,20);
                System.out.println("Area is: "+f.area());
        }
}
```

Constructor overloading: Java supports constructor overloading, which means you can have multiple constructors in the same class with different parameter lists.

Ex:

```
class Farm
{
        double length;
        double width;
        Farm(){
                length=width=0;
        }
        Farm(double x){
                length=width=x;
        }
        Farm(double length, double width)
        {
                this.length=length;
```

```
                this.width=width;
        }
        // display volume of a box
        double area() {
                return (width * length);
        }
}
class ConstructorOverloading {
        public static void main(String args[])
        {
                Farm f = new Farm();
                System.out.println("Area is: "+f.area());

                Farm f1 = new Farm(10);
                System.out.println("Area is: "+f1.area());

                Farm f2 = new Farm(20,30);
                System.out.println("Area is: "+f2.area());
        }
}
```

Output:

```
Area is: 0.0
Area is: 100.0
Area is: 600.0
```

Difference between methods and constructors:

- Constructors must have the same name as the class within which it is defined and is not same for the method in Java.
- Constructors do not return any type while method(s) have the return type or void if does not return any value.
- Constructors are called only once at the time of Object creation while method(s) can be called any number of times.

Access Control:

Java Supports access control at the class level and at the level of class members. Different types of access specifiers in JAVA are:

default	Access is permitted to any other class belonging to same package
Public	Access is permitted to any class in any package
Protected	Access is permitted to Subclass of this class in any package and to any class in the same package. The access is not permitted to any other class in another package.
Private	Access is permitted to only members of the same class.

Accessing Private Data Members:

- The private access modifier is accessible only within the same class.
- If you provide access to private data members of class to another class, that is possible with public methods of same class.

	Private	No Modifier	Protected	Public
Same class	Yes	Yes	Yes	Yes
Same package subclass	No	Yes	Yes	Yes
Same package non-subclass	No	Yes	Yes	Yes
Different package subclass	No	No	Yes	Yes
Different package non-subclass	No	No	No	Yes

Ex:

```
class Farm
{
        private double length;
        private double width;
        Farm(double x){
                length=width=x;
        }
        double getArea() {
                return (width * length);
        }
}
class AccessingPrivateMember {
        public static void main(String args[])
        {
                Farm f1 = new Farm(10);
                System.out.println("Area is: "+f1.getArea());
        }
}
```

Accessing Private Method:

- With the help of public method of same class we can access private method of the class from another class.

```
class Farm
{
        private double length;
        private double width;
        Farm(double x){
```

```
                length=width=x;
        }
        private double area(){
                return (width * length);
        }
        public double getArea() {
                return area();
        }
}
class AccessingPrivateMethod {
        public static void main(String args[])
        {
                Farm f1 = new Farm(10);
                System.out.println("Area is: "+f1.getArea());
        } }
```

Accessor methods: These are ordinary methods and are also known as setters and getter methods. These methods allow one to read and write the value of an instance variable of an object.

Recursive Methods:

In Java, recursive methods are those that call themselves in order to solve a problem.

A recursive method must have the following components:

1. **Base Case**: This is the condition that stops the recursion. It prevents infinite recursion by providing a simple, direct answer for a specific case.
2. **Recursive Case**: This is the part where the method calls itself with a modified argument, progressively working towards the base case.

Example 1: Factorial Calculation

The factorial of a non-negative integer ‘n’ is the product of all positive integers less than or equal to ‘n’. It's defined as:

- factorial(0)=1 - base case
- factorial(n)=n×factorial(n−1) for n >0 - recursive case

```
class Factorial {
   public static int factorial(int n) {
      // Base case
      if (n == 0) {
         return 1;
      }
      // Recursive case
      return n * factorial(n - 1);
   }
```

```
    public static void main(String[] args) {
        System.out.println(factorial(5)); // Output: 120
    }
}
```

Example 2: Fibonacci Sequence

The Fibonacci sequence is defined as:

- F(0)= 0 - base case
- F(1)=1 - base case
- F(n)=F(n−1)+F(n−2> n>1 - recursive case

```
class Fibonacci {
    public static int fibonacci(int n) {
        // Base cases
        if (n == 0) {
            return 0;
        }
        if (n == 1) {
            return 1;
        }
        // Recursive case
        return fibonacci(n - 1) + fibonacci(n - 2);
    }

    public static void main(String[] args) {
        System.out.println(fibonacci(5));
    }
}
```

Keywords this, final and static:

this keyword:

This keyword provides reference to the current object.
Ex:

```
import java.util.*;
class Data{
        String name;
        int age;
        double cgpa;

        void setData(String name,int age, double cgpa)
```

```
        {
                this.name=name;
                this.age=age;
                this.cgpa=cgpa;
        }
        String getData()
        {
                return name+" "+age+" "+cgpa;
        }
}
class Student
{
        public static void main(String arh[])
        {
                Scanner s=new Scanner(System.in);
                System.out.println("Enter Name , age and CGPA");
                String name=s.nextLine();
                int age=s.nextInt();
                double cgpa=s.nextDouble();

                Data d =new Data();
                d.setData(name,age,cgpa);
                System.out.println(d.getData());
        }
}
```

Another example to illustrate the use of this keyword to print the object memory address:

```
class MemoryAddressDemo1 {
        MemoryAddressDemo1(){
                        System.out.println("Inside of Constructor: " + this.hashCode());
        }
}
public class MemoryAddressDemo {
        public static void main(String[] args) {
                // Create an object
                MemoryAddressDemo1 obj = new MemoryAddressDemo1();

                // Print the object details and its hash code
                System.out.println("Object Details: " + obj);
                System.out.println("From Main method - Hash Code: " +
obj.hashCode());
        }
}
```

Output:

```
Inside of Constructor: 798154996
Object Details: MemoryAddressDemo1@2f92e0f4
From Main method - Hash Code: 798154996
```

final keyword:

- The value of a variable declared final cannot be changed in the program.
- It makes the variable a constant and must be initialized at the time of declaration.
- A few examples of declarations are as:
  final double PI = 3.14159;
  final int M = 900;
  final double X = 7.5643;
- A final variable that have no value is called blank final variable or uninitialized final variable.
- It can be initialized inside the class/ constructor only.



Ex: final variable

```
class FinalVariable {
   public static void main(String[] args) {
     // Define a constant variable PI
      final double PI = 3.14159;
      // Print the value of PI
      System.out.println("Value of PI: " + PI  );
   }
```

Another Example:

```
class  Bike
{
        final int speedlimit;
        Bike()
        {
                speedlimit=45;
        }
         void run()
         {

                 System.out.println( speedlimit );
         }
}
class Final_Demo
{
         public static void main(String args[])
         {
```

```
                Bike obj=new  Bike();
                obj.run();
        }
}
```

static keyword:

In Java, the static keyword is used to define class-level variables and methods that are associated with the class itself, rather than with instances (objects) of the class.

- **Static variable:** Static variables, also known as class variables, are shared among all instances of a class. They are initialized only once, at the start of program execution, and retain their value across all instances of the class.

```
class StaticVariable
{
        static int x=20;
}
class StaticVariableDemo
{
        public static void main(String arg[])
        {
                StaticVariable s1=new StaticVariable();
                StaticVariable  s2=new  StaticVariable();
                StaticVariable s3=new StaticVariable();

                System.out.println(StaticVariable.x);
                System.out.println(s1.x);
                System.out.println(s2.x);
                System.out.println(s3.x);

                s1.x=200;

                System.out.println(s1.x);
                System.out.println(s2.x);
                System.out.println(s3.x);
        }
}
```

```
O/P:
20
20
20
20
200
200
200
```

- **Static method:** Static methods belong to the class rather than any specific instance. They can be called without creating an instance of the class. Static methods can only access other static members (variables and methods) of the class.

```
class StaticMethod
{
        static int x;
        static void method(int x1)
        {
                x=x1;
        }
        static void access()
        {
                method(2);
                System.out.println(x );
        }
}
class StaticMethodDemo
{
        public static void main(String arg[])
        {
                StaticMethod sm=new StaticMethod();
                StaticMethod.access();//accessing with class name
                sm.access(); //accessing with instance
        }
}
```

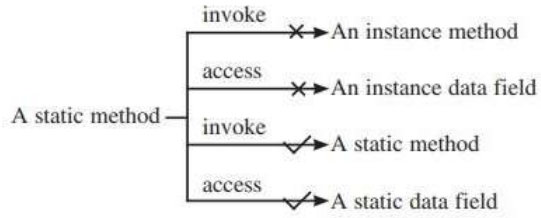
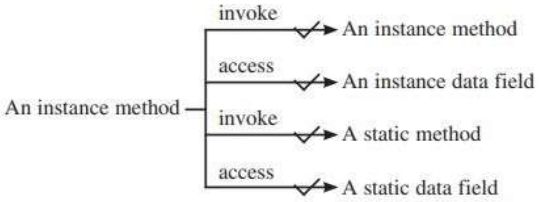
```
O/P:
2
2
```

- **Static block:** Static blocks are used for initializing static variables. They are executed once when the class is first loaded, before any static methods or variables are accessed.

```
class StaticBlock1
{
        static{
                System. out.println("Static Block");
        }
        public static void main(String arg[])
        {
                System. out.println("Main Method");
        }
}
```

```
O/P:
Static Block
Main Method
```

Differences between static and instance methods:

Static Methods	Instance Methods
Can be called without the objects,	Require an object creation to call an instance method
Associated with the class	Associated with the objects
Can access only static attributes.	Can access all attributes of a class
Requires to be declared using static keyword	Doesn't require any keyword
	

Passing Arguments by Value and by Reference

There are two ways to pass the arguments in JAVA.

i. Pass by value

ii. Pass by reference

Pass by value: When you pass a primitive type (such as int, char, float, etc.) to a method, Java passes a copy of the value. This means that changes made to the parameter within the method do not affect the original value.

Ex:

```
class SwapWithTemp {
   static void swap(int a, int b) {
      int temp = a;
      a = b;
      b = temp;

      System.out.println("After swapping: a = " + a + ", b = " + b);
   }
   public static void main(String[] args) {
      int x = 5;
      int y = 10;
      System.out.println("Before swapping: x = " + x + ", y = " + y);
      swap(x, y);
   }
}
```

Output:

Before swapping: x = 5, y = 10
After swapping: a = 10, b = 5

Pass by reference: When you pass an object reference to a method, Java also passes the reference by value. This means that the method receives a copy of the reference, not the actual object. Both the original reference and the copied reference point to the same object in memory.

```
import java.util.Scanner;
class Question {   //Define a class Question with two elements e1 and e2.
        Scanner sc = new Scanner(System.in);
        int e1 = sc.nextInt();  //Read e1
        int e2 = sc.nextInt();  //Read e2
}
class SwappingWithRef {
// Define static method swap()to swap the values of e1 and e2 of class Question.
                static void swap(Question t1)
                {
                        int temp;
                        temp=t1.e1;
                        t1.e1=t1.e2;
                        t1.e2=temp;
                }
                public static void main(String[] args) {
                        Question t = new Question();
                        System.out.println("Before Swap: " + t.e1 + " " + t.e2);
                        swap(t);
                        System.out.print("After Swap: " + t.e1 + " " + t.e2);
                }
}
```

Output:

10
20
Before Swap: 10 20
After Swap: 20 10

Passing object as a reference to the constructor example program:

```
class Farm
{
        double length;
        double width;

        Farm(double x){
                length=width=x;
        }
        Farm(Farm f)
        {
                 length=f.length;
                 width=f.width;
```

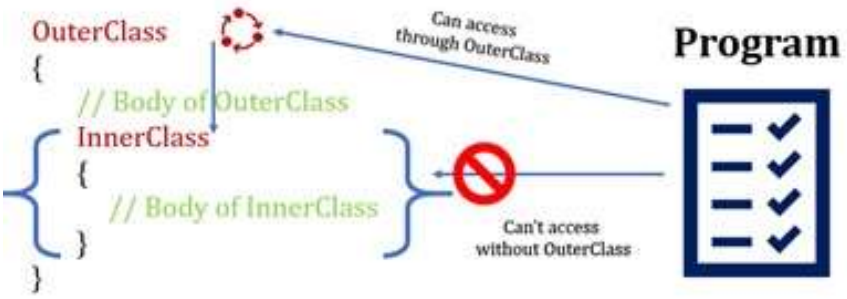
```
        }
        double area() {
                return (width * length);        }
}
class ConstructorPassingObject {
        public static void main(String args[])
        {
                Farm f1 = new Farm(10);
                System.out.println("Area is: "+f1.area());
                Farm f2 = new Farm(f1);
                System.out.println("Area is: "+f2.area());
        } }
```

Nested Classes:

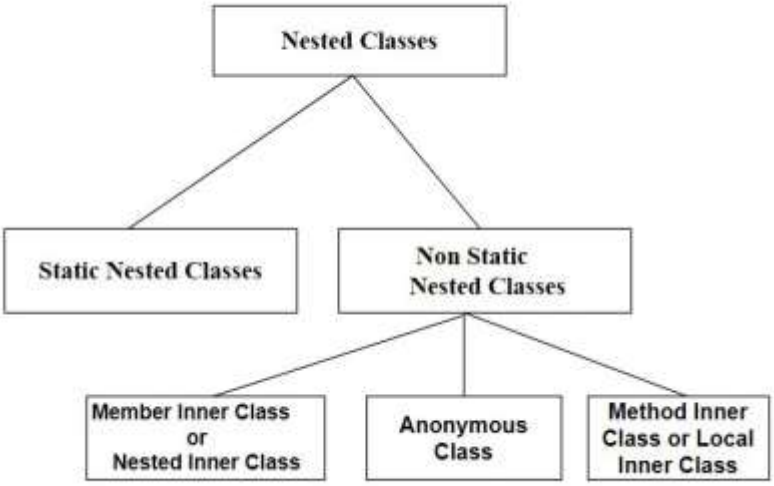
- A class defined inside another class is called a Nested Class**.**
- The class that contains the other class is known as the outer class and the contained class is known as inner class.

```
OuterClass
{
        // Body of OuterClass
        InnerClass
        {
                // Body of InnerClass
        }
}
```

- The class which is nested, exists only as long as the enveloping class exists.
- So, the scope of inner class is limited to the scope of enveloping class.



- Nested classes enables us to logically group classes that are only used in one place, thus this increases the use of encapsulation.
- Nested classes are divided into two categories:
    - static nested class: Nested classes that are declared static are called static nested classes.
    - Nested non-static classes or inner classes which may be member inner class, local class and anonymous class.



Static Nested Class:

- In Java, a static nested class is a nested class that is declared with the static keyword.

```
class Outer {
   static class Inner {
      void display() {
         System.out.println("Hello from Inner");
      }
   }
}
```

- Static classes can be accessed with outer class name.

```
public class Test {
   public static void main(String[] args) {

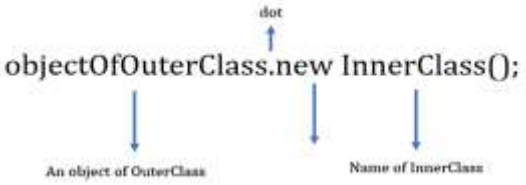
      Outer.Inner obj = new Outer.Inner();
      obj.display();
   }
}
```

Member Inner Class or Nested Inner Class:

- If the nested class is **not static**, it is called an **Inner Class**.

```
class OuterClass
{
    // A member inner class named Inner.
    class InnerClass
    {
        .........
    }
}
```

- We can't create an object or instance of a member inner class directly. The object/instance of a member inner class totally depends on the object of the outer class.



OuterClass objOfOuter = new OuterClass();
OuterClass .InnerClass objOfInner = objOfOuter.new InnerClass();

Ex:

```
class Outer {
   int x = 100;

   class Inner {
      void display() {
         System.out.println("x = " + x);
      }
   }
}
```

To create the Inner class object:

```
public class Test {
   public static void main(String[] args) {

      Outer outer = new Outer();
      Outer.Inner inner = outer.new Inner();
      inner.display();
   }
}
```

Anonymous class:

- A class that has no name is known as an anonymous inner class in Java.

Without Anonymous Inner Class

```
interface Animal {
   void sound();
}
class Dog implements Animal {
   public void sound() {
      System.out.println("Dog barks");
```

```
    }
}
public class Test {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    } }
```

Using Anonymous Inner Class

- Now suppose we need a **Dog only once**.
- We don't want to create a separate `Dog` class.
- We can directly write:

```
interface Animal {
    void sound();
}

public class Test {
    public static void main(String[] args) {

    Animal a = new Animal() {
            public void sound() {
                System.out.println("Dog barks");
            }

        };

        a.sound();
    }
}
```

Method Inner Class or Local Inner Class:

- A class i.e., created inside a method or a block, is called local inner class in Java.
- Block may be *method or if clause or loop.*

```
class OuterClass {
    void outerClassMethod() {

        // Local Inner Class
        class InnerClass {

            public void innerClassMethod() {
                System.out.println("Method of inner class");
            }
        }
```

```
        // Creating object of Local Inner Class
        InnerClass obj = new InnerClass();
        obj.innerClassMethod();
    }

    public static void main(String args[]) {

        OuterClass outer = new OuterClass();
         outer.outerClassMethod();
    }
}
```

String Handling in JAVA:

Introduction:

A string is a sequence of characters. In Java, string is an object of class, and therefore there is no automatic appending of null character by the system.

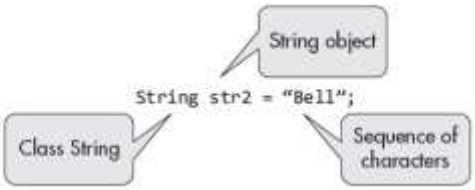
- In Java, there are three classes that can create strings and process them with nearly similar methods.
    - class String
    - class StringBuffer
    - class StringBuilder
- All the three classes are part of java.lang package.

Class String:

In the case of String class, an object may be created as

String str1=”abcd”;

String str2=”bell”;



and these are equavelent to char s1={‘a’,’b’,’c’,’d’} and char s2={‘b’,’e’,’l’,’l’}

Objects of class String may also be created by operator new

String str=new String(“abcd”);

In the case of other two classes, the objects can be created only by the operator new as

StringBuffer sb=new StringBuffer(“PVPSIT”);

StringBuilder sb=new StringBuilder(“PVPSIT”);

Storage of strings:

- The objects of class String have a special storage facility, which is not available to objects of other two String classes or to objects of any other class.

- The memory allocated to a Java program is divided into two segments:

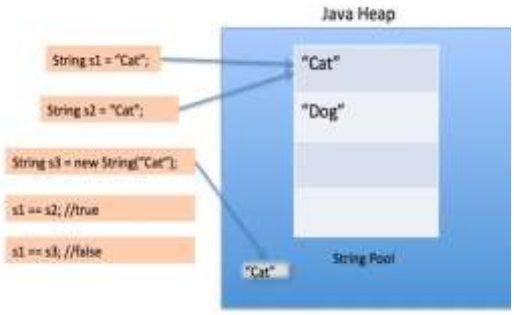
  (i) Stack (ii) Heap

- Within the heap, there is a memory segment called 'String constant pool'.
- The String class objects can be created in two different ways:

  String s1 = "Cat";

  String strz = new String("Cat");

Ex:




```
class String_Pool
{
        public static void main(String arg[])
        {
                String s1="Cat";
                String s2= "Cat";

                String s3=new String("Cat");
                if(s1==s2)
                        System.out.println("true");
                else
                        System.out.println("false");

        }
}
```

The strings that are defined like String s1="Cat" are in the String constant pool, whereas the strings that are defined by operator new are stored in the heap memory (outside of string constant pool) with the objects of other classes.

Immutability:

- The string objects created by class String are immutable.
- By immutable implies that once an object is created, its value or contents cannot be changed.
- Neither the characters in the String object once created nor their case (upper or lower) can be changed.
- New String objects can always be created and assigned to older String object references.
- Thus, when you change the content of a string by defining a new string, the old and new remain in the memory.

- The immutable objects are thread safe and so are the String objects.
- The objects created by class StringBuffer are mutable.
- These are stored on the heap segment of memory outside the String constant pool.
- The contents of StringBuffer strings may be changed without creating new objects.
- The methods of StringBuffer are synchronized, and hence, they are thread safe.
- The objects of class StringBuilder are also mutable but are not thread safe.
- The operations are fast as compared to StringBuffer and there is no memory loss as is the case with String class.
- The class StringBuilder has the same methods as the class StringBuffer.
- Therefore, if multithreads are not being used, then StringBuilder class should be used to avoid memory loss.
- Properties of String, StringBuffer and StringBuilder classes:

Property	String	StringBuffer	StringBuilder
Storage area	Constant string pool and heap	Heap	Heap
Object mutability	Not mutable	Mutable	Mutable
Thread safety	Safe	Safe	Not safe
Performance	Fast	Slow	Fast

Interface CharSequnece:

- It is an interface in java.lang package.
- It is implemented by several classes including the classes String, StringBuffer, and StringBuilder.
- It has the following four methods.

Modifier	Method Name
char	charAt(int index): Returns the char value at the specified index.
int	length(): Returns the length of this character sequence.
CharSequence	subSequence(int start, int end): Returns a new CharSequence that is a subsequence of this sequence.
String	toString(): Returns a string containing the characters in this sequence in the same order as this sequence.

Ex:

```
class CharSeq{
        public static void main(String ar[]){
                CharSequence str="PVPSIT";
                System.out.println("Use of charAt():"+str.charAt(1));
                System.out.println("Use of length():"+str.length());

                CharSequence str1=new StringBuffer("AMARAVATI");
                CharSequence str2=new StringBuilder("ANDHRA");

                CharSequence s =str1.subSequence(2,6);//from 2 to 6-1
                System.out.println("str1:"+str1);
                System.out.println("str2:"+str2);
                System.out.println("s:"+s);
        }
}
```

```
Use of charAt():V
Use of length():6
str1:AMARAVATI
str2:ANDHRA
s:ARAV
```

Class String:

- The class String is used to represent strings in Java. It is declared as

  public final class String extends Object implements serializable, comparable<String>, charSequence

- The String class is final, it cannot be extended.

Constructors: There are number of constructors of class String, and therefore strings can be created in a variety of ways.

1. ***String()*** - This constructor creates a string without any characters.

   String str = new String();

   String str1 = “”;

2. ***String(byte[] barray)*** – It constructs a new string by decoding the specified byte[] array using computers default character set(ASCII).

   byte []barray = new byte[]{65, 66, 67, 68, 69};

   String str2 = new String(barray); // str2 = “ABCDE”.

3. ***String (byte [] barray, Charset specifiedset)*** - It constructs a new string by decoding the specified byte array (bray) by using specified character set. Some of the Charsets supported by Java are UTF8, UTF16, UTF32, and ASCII. These may be written in lower case such as utf8, utf16, utf32, and ascii.

   byte []barray = new byte[]{65, 66, 67, 68, 69};

String str=new String(barray, “ascii”);

4. ***String(****byte[] byte_arr, int start_index, int length****)*** **-** Construct a new string from the bytes array depending on the start_index(Starting location) and length(number of characters from starting location).

```
byte[] b_arr = {71, 101, 101, 107, 115};
String s = new String(b_arr, 1, 3); // eek
```

5. **String(byte[] byte_arr, int start_index, int length, Charset char_set) -** Construct a new string from the *bytes array* depending on the *start_index(Starting location)* and *length(number of characters from starting location)*. Uses *char_set* for decoding.

```
byte[] b_arr = {71, 101, 101, 107, 115};
String s = new String(b_arr, 1, 4, "US-ASCII"); // eeks
```

6. ***String(char[] value)* –** Constructs the string based on the specified character array.

```
char char_arr[] = {'S', 'u', 'r', 'e', 's', 'h'};
String s = new String(char_arr); //Suresh
```

7. **String(char[] char_array, int start_index, int count) -** Allocates a String from a given *character array* but choose *count* characters from the *start_index*.

```
char char_arr[] = {'G', 'e', 'e', 'k', 's'};
String s = new String(char_arr , 1, 3); //eek
```

8. **String(int[] uni_code_points, int offset, int count) -** Allocates a String from a *uni_code_array* but choose *count* characters from the *start_index*.

```
int[] uni_code = {71, 101, 101, 107, 115};
String s = new String(uni_code, 1, 3); //eek
```

9. **String(StringBuffer s_buffer) -** Allocates a new string from the string in *s_buffer.*

```
StringBuffer s_buffer = new StringBuffer("Geeks");
String s = new String(s_buffer); //Geeks
```

10. **String(StringBuilder s_builder) -** Allocates a new string from the string in *s_builder*

```
StringBuilder s_builder = new StringBuilder("Geeks");
String s = new String(s_builder); //Geeks
```

Ex:

```
class StringConstructor
{
        public static void main(String arg[])throws Exception{
                String str1=new String();
```

```
        byte []barray=new byte[]{65,66,67,68,69,70,71,72};
        byte []uarray=new byte[]{'\u0041','\u0042','\u0043'};

        String str2=new String(barray);
        String str3=new String(barray,1,3);

        String str4=new String(barray,"ascii");

        String str5=new String("Amaravati");
        String str6=new String(uarray);

        System.out.println("String 1:"+str1);
        System.out.println("String 2:"+str2);
        System.out.println("String 3:"+str3);
        System.out.println("String 4:"+str4);
        System.out.println("String 5:"+str5);
        System.out.println("String 6:"+str6);
    }
}
```

```
String 1:
String 2:ABCDEFGH
String 3:BCD
String 4:ABCDEFGH
String 5:Amaravati
String 6:ABC
```

Methods for Extracting Characters from Strings:

Method	Description
char charAt(int index)	returns char value for the particular index
int length()	returns string length
void getChars(int start, int end, char[] destination_array, int destination_array_position)	Extracts more than one character at a time the string and copies into destination array at specified location.
byte[] getBytes()	Encodes a given string into a sequence of bytes and return an array of bytes
char toCharArray()	Converts the string into an array of characters and returns the array.

Ex:

```
class ExtractionCharacters_Strings {
  public static void main(String[] args) {
        String str="PVPSIT";
        System.out.println("Use of charAt(): "+str.charAt(1));
        System.out.println("Use of length(): "+str.length());

        char[] myArray = {'0', '1', '2', '3', '4', '5', '6', '7', '8', '9'};
        System.out.println(myArray);

          String myStr = "Hello, World!";
          myStr.getChars(7, 12, myArray, 4);
```

```
O/P:
Use of charAt(): V
Use of length(): 6
0123456789
0123World9
80 86 80 83 73 84
```

```
            System.out.println(myArray);

            byte[] result = str.getBytes();
            for(int i=0;i<str.length();i++)
                System.out.print(result[i]+" ");
        }
    }
```

Methods for comparision of Strings:

int compareTo(String str)	is used for comparing two strings lexicographically. Each character of both the strings is converted into a Unicode value for comparison. If both the strings are equal then this method returns 0 else it returns positive or negative value.
int compareToIgnoreCase (String str)	comparison of two strings irrespective of cases. Returns the value same as above.
boolean equlas()	Compares the characters inside the two strings. It is different from ==(it checks references), where as equals() checks content.
boolean equalsIgnoreCase(String another)	Compares with another string. It doesn't check case. Returns boolean value.
boolean startsWith(String chars)	The startsWith() method checks whether a string starts with the specified character(s).
boolean endsWith(String chars)	The endsWith() method checks whether a string ends with the specified character(s).
public boolean regionMatches(int toffset, String other, int ooffset, int len)	It compares a selected portion of one String with a selected portion of another String.
public boolean regionMatches(boolean ignoreCase, int toffset, String other, int ooffset, int len)	

Ex:

```
class Comparision_Strings {
  public static void main(String[] args) {
        String str1=new String("PVPSIT");
        String str2=new String("pvpsit");

        System.out.println("Use of compareTo(): "+str1.compareTo(str2));
```

```
Use of compareTo(): -32
Use of compareToIgnoreCase(): 0
Use of equals(): false
Use of equalsIgnoreCase(): true
use of endsWith(str):true
use of startsWith(str):true
false
true
```

```
        System.out.println("Use of compareToIgnoreCase():
"+str1.compareToIgnoreCase(str2));

        System.out.println("Use of equals(): "+str1.equals(str2));
        System.out.println("Use of equalsIgnoreCase(): "+str1.equalsIgnoreCase(str2));

        System.out.println("use of endsWith(str):"+str1.endsWith("SIT"));
        System.out.println("use of startsWith(str):"+str1.startsWith("PVP"));

        String myStr1 = "Hello, World!";
         String myStr2 = "New WORLD";
         System.out.println(myStr1.regionMatches(7, myStr2, 4, 5));
         System.out.println(myStr1.regionMatches(true,7, myStr2, 4, 5));
  }
}
```

Methods for Modifying Strings:

In Java, string is an object of class String, and therefore it cannot be modified. The ways to modify strings are:

1. Define a new string
2. Create a new string using substring() from existing string
3. Creating new string by adding two or more substrings

Method	Description
String substring(int beginIndex)	Returns substring from the begin index to last character
String substring(int beginIndex, int endIndex)	Returns substring from the begin index to end index-1
String concat(String str)	concatenates specified string with existing one.
String replace(char searchChar, char newChar)	The replace() method searches a string for a specified character, and returns a new string where the specified character(s) are replaced
String replace(char searchCharSeq, char newCharSeq)	Returns new string in which all occurences of a string of characters is replaced with another sequence of characters.
String trim()	The trim() method removes whitespace from both ends of a string.

Ex:

```
class ModifyingStrings
{
        public static void main(String arg[])
        {
                String s1="PVP Siddhartha";
```

```
Concatenation of first and second      :PVP
SiddharthaEngineering
Extraction of substrings               :P Siddhartha
Extraction of substrings               :P S
Use of Trim()              :Hello
Use of replace():AVASIT
Use of replace():APSIT
```

```
        String s2=new String("Engineering");

        System.out.println("Concatenation of first and second        :"+s1.concat(s2));

        System.out.println("Extraction of substrings          :"+s1.substring(2));
        System.out.println("Extraction of substrings          :"+s1.substring(2,5));

        String str = "      Hello     ";
        System.out.println("Use of Trim()                     :"+str.trim());

        String s3 = "PVPSIT";
        System.out.println("Use of replace():"+s3.replace('P','A'));
        System.out.println("Use of replace():"+s3.replace("PV","A"));
    }
}
```

Methods for Searching Strings:-

Method	Description
int indexOf(String str/char ch)	The indexOf() method returns the position of the first occurrence of specified character(s) in a string.
int indexOf(String str/char ch, int fromIndex)	The indexOf() method returns the position of the first occurrence of specified character(s) in a string. Search starts from specified location.
int lastIndexOf(String str/char ch)	The lastIndexOf() method returns the position of the last occurrence of specified character(s) in a string.
int lastIndexOf(String str/char ch, int fromIndex)	The lastIndexOf() method returns the position of the last occurrence of specified character(s) in a string. Search starts from specified location.

Ex:

```
class SearchString {
  public static void main(String[] args) {
        String myStr = "Amaravati";
        System.out.println(myStr.indexOf("a", 2));  //  - returns 2
        System.out.println(myStr.indexOf("av", 2)); //  - returns 4
        System.out.println(myStr.lastIndexOf("av"));//  - returns 4
        System.out.println(myStr.lastIndexOf("a"));//  - returns 6
  }
}
```

Differences between == and equals(): ==checks the references, where as equals() method check the content.

```
class CompareOperator
{
	public static void main(String arg[])
	{
		String s1="Hello";
		String s2=new String("Hello");

		if(s1==s2)
			System.out.println("Same");
		else
			System.out.println("Not Same");
	}
}
```

Output: Not Same

Explanation: When an object is created by JVM, it returns the memory address of the object as a hexadecimal number, which is called object reference. Whenever, a new object is created, a new reference numbers is allotted to it. It means every object will have a unique reference.

So, to compare two strings use **equals()** method.

```
class CompareEqual
{
	public static void main(String arg[])
	{
		String s1="Hello";
		String s2=new String("Hello");

		if(s1.equals(s2))
			System.out.println("true");
		else
			System.out.println("false");

	}
}
```

Output: true

Difference between == and equals() while comparing strings: == operator compares the reference of the string objects, not the content of the objects. Where as equals method compares the contents of the strings not object references.

Class StringBuffer:

A String class objects are immutable and hence their contents cannot be modified. StringBuffer class objects are mutable, so they can be modified. Moreover the methods that directly manipulate the data of the object are not available in String class. Such methods are available in StringBuffer class.

Constructors:

1. StringBuffer(): It reserves room for 16 characters without reallocation

```
StringBuffer s = new StringBuffer();
```

2. StringBuffer(int size): It accepts an integer argument that explicitly sets the size of the buffer.

```
StringBuffer s = new StringBuffer(20);
```

3. StringBuffer(String str): Creates a StringBuffer containing the specified String and provides capacity for **16 additional characters.**

```
StringBuffer s = new StringBuffer("PVPSIT");
```

Methods:

Sl. No	Method	Description
1	append(String s)	is used to append the specified string with this string. The append() method is overloaded like append(char), append(boolean), append(int), append(float), append(double) etc
2	insert(int offset, String s)	is used to insert the specified string with this string at the specified position.
3	replace(int startIndex, int endIndex, String str)	is used to replace the string from specified startIndex and endIndex.
4	delete(int startIndex, int endIndex)	is used to delete the string from specified startIndex and endIndex
5	reverse()	is used to reverse the string
6	ensureCapacity(int minimumCapacity)	is used to ensure the capacity at least equal to the given minimum.
7	charAt(int index)	is used to return the character at the specified position.
8	length()	is used to return the length of the string i.e. total number of characters
9	String substring(int beginIndex)	is used to return the substring from the specified beginIndex.
10	substring(int beginIndex, int endIndex)	is used to return the substring from the specified beginIndex and endIndex.

Example Program:

```
class StringBufferDemo
{
        public static void main(String arg[])
        {
                StringBuffer sb=new StringBuffer();
                System.out.println("Use of capacity():"+sb.capacity());

                StringBuffer sb1=new StringBuffer("Uni");
                System.out.println("Use of append():"+sb1.append("versity"));
                System.out.println("Use of insert():"+sb1.insert(0, "JNTU "));
                System.out.println("use of delete()"+sb1.delete(0,4));
                System.out.println("Use of length():"+sb1.length());

                System.out.println("Use of reverse():"+sb1.reverse());
                System.out.println("Use of reverse():"+sb1.reverse());

                System.out.println("Use of replace():"+sb1.replace(0,3,"ANU "));


                StringBuffer sb4=new
                StringBuffer("AAAAAAAAAAAAAAAAAAAAAAAAAAAA");
                System.out.println("Use of capacity():"+sb4.capacity());
        }
  }
```

Output:

```
Use of capacity():16
Use of append():University
Use of insert():JNTU University
use of delete() University
Use of length():11
Use of reverse():ytisrevinU
Use of reverse(): University
Use of replace():ANU iversity
Use of capacity():44
```

Use of final method:

- We use a final method when we want to **protect the original implementation** of a method.
- A final method is used to prevent its overriding by subclasses and to preserve its original implementation.

```
class Bank {

   final void rules() {
      System.out.println("Bank rules");
   }
}
```

- If different bank subclasses are created:

```
class SBI extends Bank {
   // Cannot change rules()
}

class HDFC extends Bank {
   // Cannot change rules()
}
```

Declaration	Effect
final variable	Cannot change the value
final method	Cannot override the method
final class	Cannot extend the class

Object class**:**

- It is the **root/superclass of all classes**.
- Every class in Java **directly or indirectly inherits from Object**.

Ex:

```
class Student {
   int marks = 90;
}
```

Java internally considers it approximately as:

```
class Student extends Object {
   int marks = 90;
}
```

Important methods of Object class

Method	Use
toString()	Returns string representation of an object
equals()	Compares objects
hashCode()	Returns hash code of an object
getClass()	Returns the runtime class of an object
clone()	Creates a copy of an object (subject to conditions)
finalize()	**Deprecated/removed from modern Java use**;

Ex:

```
class Student {
   int marks = 90;
}
public class ObjectClass  {
   public static void main(String[] args) {
      Student s = new Student();

      System.out.println(s.toString());
      System.out.println(s.getClass());
      System.out.println(s.hashCode());
   }
}
```

Output:

```
Student@4517d9a3
class Student
1159190947
```

Write a Java program to find longest word in given input.

```
import java.util.Scanner;

public class W07_P1 {
//code for print the longest word.
        public static String findLongestWord(String str) {

            str+=" ";
            String word = "", lWord = "";
            for (int i = 0; i <  str.length(); i++)
            {
               char ch = str.charAt(i);
               if (ch == ' ')
               {

                   if (word.length() > lWord.length())
                      lWord = word;

                    word = "";
               }
               else {
                  word += ch;
               }
            }
             return lWord;
   }
public static void main(String[] args) {
      Scanner scanner = new Scanner(System.in);

      // Prompt user to enter text
     // System.out.println("Enter some text:");
      String text = scanner.nextLine();

      // Close the scanner
      scanner.close();

      // Call the method to find the longest word
      String longestWord = findLongestWord(text);

      // Print the longest word found
      System.out.print("The longest word in the text is: " + longestWord);
   }
}
```

Explain what is meant by public static void main(String arg[])

- **public**: it is access modifier: The public keyword means that the main method is accessible from anywhere. This is necessary because the JVM needs to call this method from outside the class, so it must be publicly accessible.
- **static**: The static keyword means that the method belongs to the class itself rather than to instances of the class. Since the main method is called by the JVM before any objects of the class are created, it must be static.
- **void**: is a return type: The void keyword indicates that the main method does not return any value. The main method is designed to execute code and does not need to return any result.
- **main**: is a method name: main is the name of the method that the JVM looks for to start the execution of a Java program. This name is fixed and must be exactly main.
- **String[] args**: String[] args is an array of String objects that can hold command-line arguments passed to the program. This allows you to pass information to the application when it starts. If no command-line arguments are provided, the array will be empty but not null.

Explain System.out.println()?

Part	Meaning
System	A predefined class in the java.lang package.
out	A static object of the PrintStream class that represents the standard output (console).
println()	A method of the PrintStream class that prints data and moves the cursor to the next line.